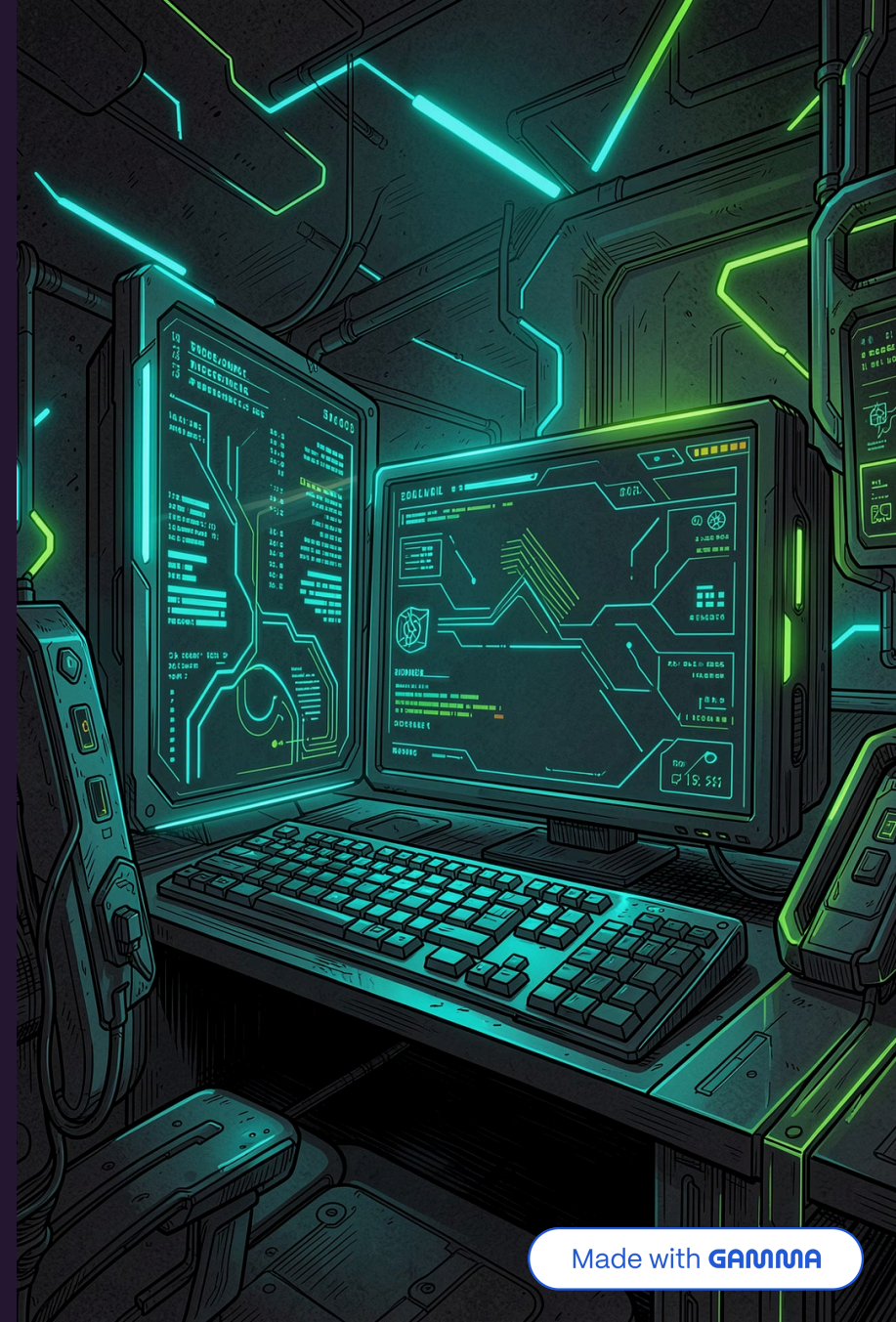


Arquitectura y Concurrency en Sistemas de Chat (Go)

Presentación técnica: diseño arquitectónico de un sistema de chat en tiempo real implementado en Go. Tema: Dark Mode (gris carbón / negro profundo) con acentos en verde neón y cian eléctrico. Público: equipos técnicos y profesor universitario — enfoque en decisiones arquitectónicas y concurrency.



Agenda

1. Filosofía del diseño

Por qué DDD en Go y separación de responsabilidades

2. Concurrencia y Hub/Client

Goroutines, Channels y escalabilidad

3. Seguridad

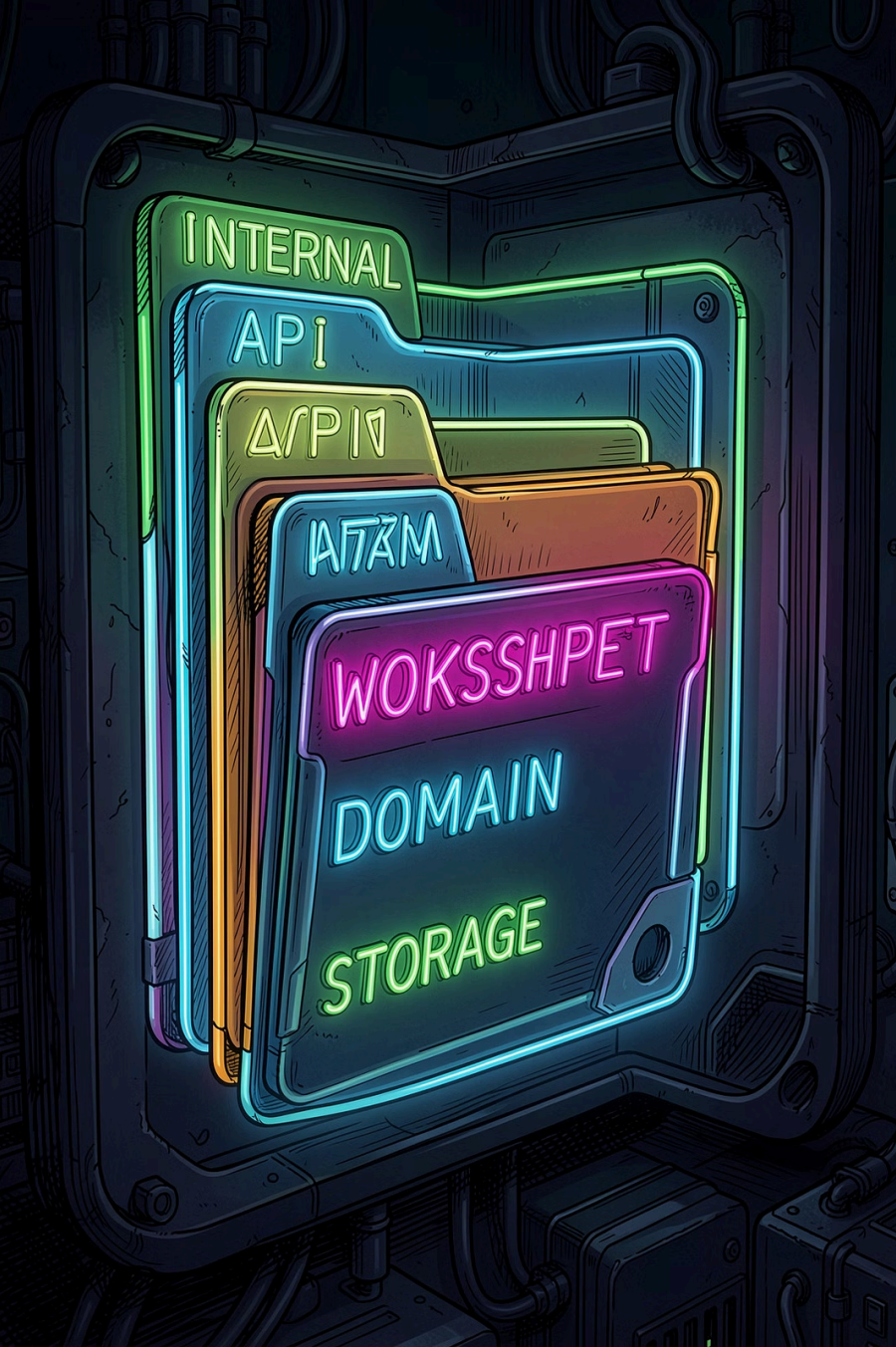
Elección de bcrypt y parámetros de trabajo

4. Metodología

Git Flow y organización de paquetes

5. Demo / Preguntas

Guion de exposición y justificación técnica

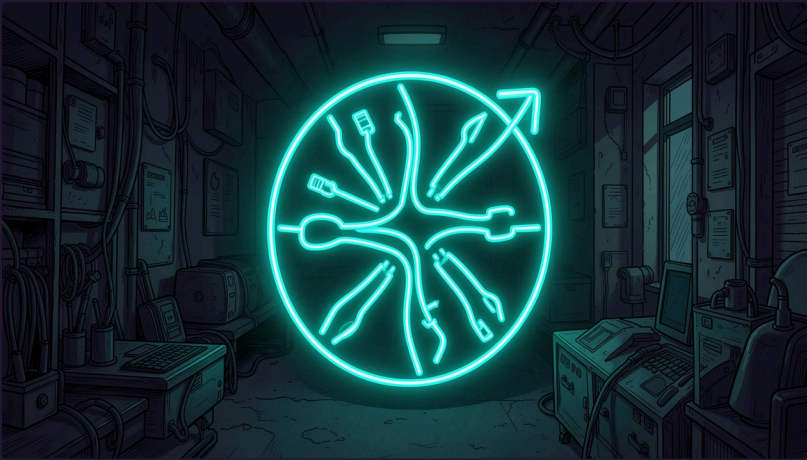


Filosofía del Diseño: DDD aplicado y límites de OOP

Transición: abandonar la mentalidad de objetos ricos y herencia profunda para favorecer límites explícitos (bounded contexts) y paquetes con responsabilidades claras. En Go esto se traduce en:

- internal/: aislamiento de la lógica del dominio frente a la API pública.
- packages pequeños y composables — interfaces finas para invertir dependencias.
- tests por contrato (blackbox) para reemplazo de infra (DB, brokers) sin cambios en el dominio.

Beneficio: despliegues y cambios de infra con coste mínimo en la lógica de negocio.



El corazón: Goroutines y Channels

Diseño concurrente nativo: Goroutines para unidades de trabajo; Channels para comunicación segura sin locks. Principios clave:

- Preferir passing-by-channel sobre estado compartido cuando sea posible.
- Diseñar workers y pools para operaciones costosas (DB, persistencia de mensajes).
- Context.Context para cancelación y límites temporales de petición.

Patrón Hub / Client para WebSockets

Arquitectura de runtime:

1

Hub – único orquestador

Registra/deregistra clients; despacha broadcast y rooms; mantiene métricas de conexión.

2

Client – conexión por socket

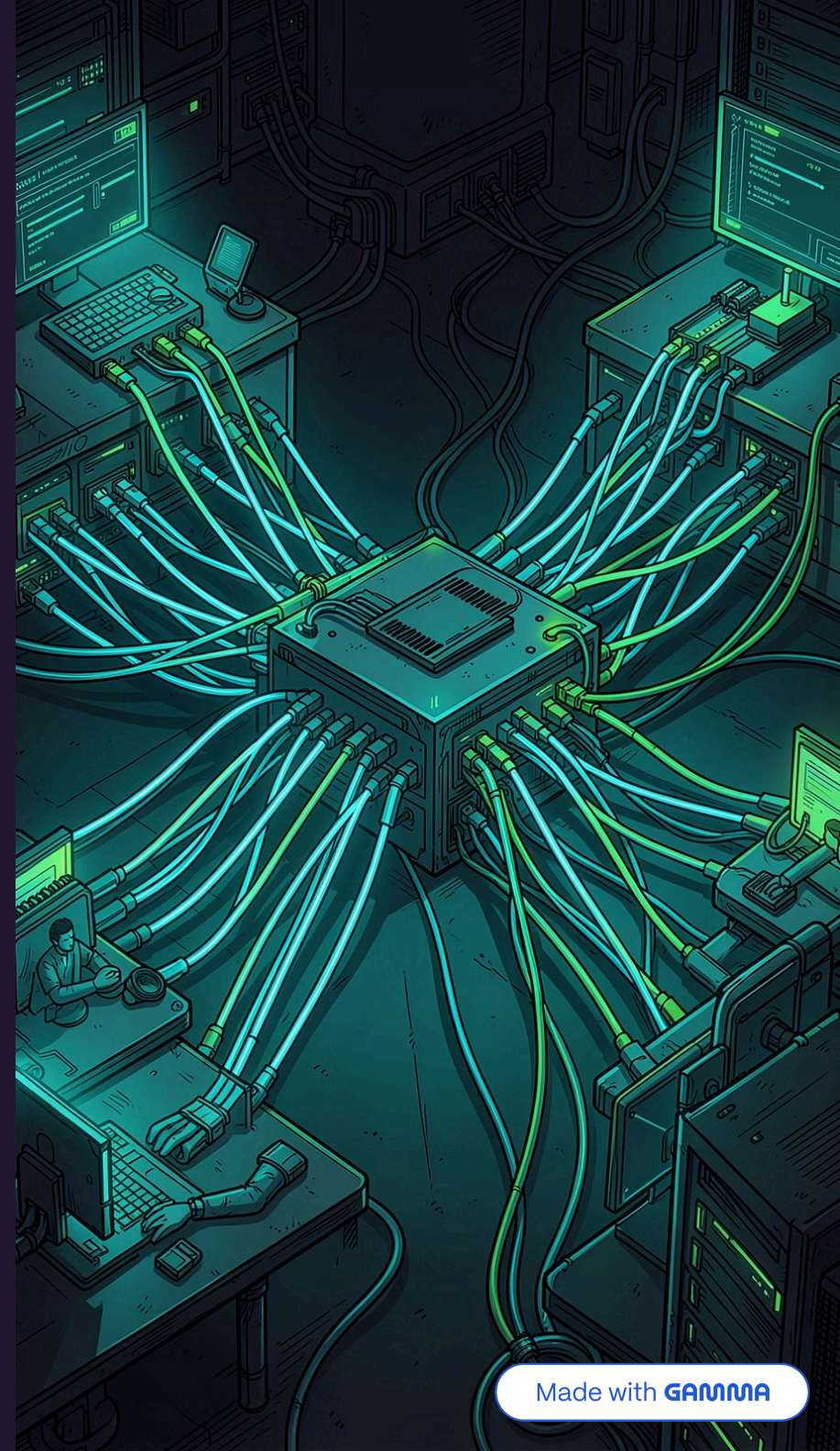
Goroutine de lectura, goroutine de escritura, buffer selectivo y backpressure.

3

Channels de control

Channels tipados para mensajes, comandos de control y señales de cierre; evitar goroutine leaks cerrando canales con select.

Escalado horizontal: sticky sessions + balanceador + réplica del Hub con coordinación vía pub/sub (e.g., NATS, Redis Streams).





Escalabilidad y tolerancia

Modelo de despliegue recomendado:

- Instancias stateless del servicio de conexión detrás de un load balancer L4.
- Coordinación de presencia y distribución de mensajes mediante un bus ligero (NATS o Redis Streams) para replicar eventos entre hubs.
- Observabilidad: métricas (Prometheus), tracing (OpenTelemetry) y logging estructurado al estilo terminal neon.

Seguridad Crítica: por qué bcrypt

Decisión técnica: bcrypt como única estrategia de hashing de contraseñas en el servicio de autenticación.

- Adaptabilidad: work factor configurable para aumentar coste computacional con el tiempo.
- Resistencia a ataques por fuerza bruta y hardware especializado (comparado con MD5/SHA sin sal).
- Implementación en Go robusta y auditada — uso de salt embebido y gestión de versiones de hash.

⚠ Evitar: almacenamiento de contraseñas en texto plano o con funciones rápidas. No reinventar algoritmos; usar librerías mantenidas y revisar parámetros periódicamente.



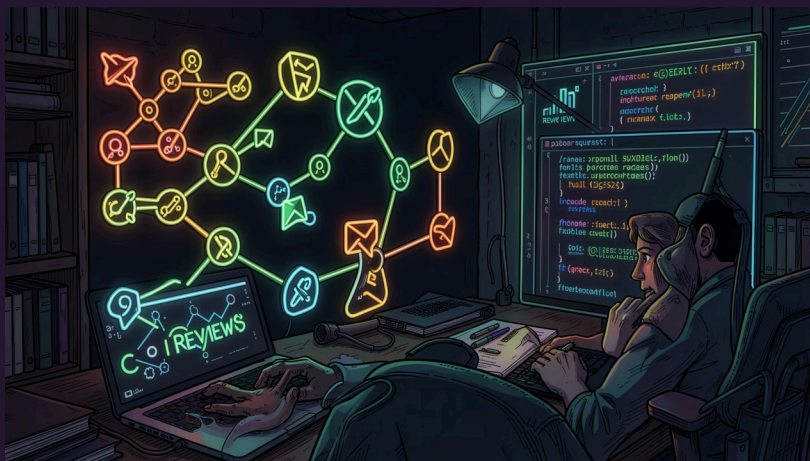
Metodología de Equipo y Git Flow

Flujo estándar aplicado al proyecto:

- Ramas: main (estable), develop (integración), feature/*, release/*, hotfix/*.
- PRs obligatorios con revisión: lint, tests unitarios y de integración (WebSocket emulación).
- CI/CD: pipelines para build, tests y despliegue a staging; gates para performance y seguridad.

Organización de paquetes (convención sugerida):

1. cmd/ — ejecutables
2. internal/ — dominio, usecases, delivery (websocket)
3. pkg/ — utilidades reutilizables
4. infra/ — adaptadores: storage, broker, auth



Organización de Paquetes: ejemplo práctico



internal/domain

Entidades, Value Objects y reglas de negocio. Sin dependencias externas.



internal/delivery

Adaptadores HTTP y WebSocket — traduce DTOs a modelos de dominio.



infra

Implementaciones concretas: bcrypt, DB, pub/sub. Inyectables para tests.